

EGE YESILYURT

Richmond Hill, ON (Greater Toronto Area) | 647-804-5834 | contact@egeyesilyurt.ca
egeyesilyurt.ca | github.com/egeyesil | linkedin.com/in/egeyesil

Summary

Software Engineering student shipping full-stack apps in **Python**, **TypeScript** and **Java**: **React**/Next.js frontends, **REST APIs**, **PostgreSQL** (hand-written **SQL**, **migrations**, **query optimization**). Applies **AI/ML** in production (**LLM tool-calling agent**, **constraint-programming** generator, tooling for **AI coding agents**); ships with **Docker** and **CI/CD**.

Education

York University, North York, ON

Expected Late 2027

Bachelor of Engineering, Major in Software Engineering

Relevant Coursework: Object-Oriented Programming (Java), Data Structures and Algorithms, Operating Systems, Software Design, Introduction to Database Systems, **Applied Machine Learning** (PyTorch, scikit-learn, NLP, RAG)

Projects

z9bra: Daily Logic Puzzle with a Constraint-Solver Generator | [Website](#) | [Source Code](#)

Live, June 2026

- Shipped a Wordle-style daily logic puzzle in **Next.js/React** and **TypeScript** on Vercel, with date-seeded selection so every player gets the same grid on a fixed Toronto reset and **Satori**-rendered share cards; reached **200+ unique players** and a peak of 60 daily active players.
- Stored account-synced streaks in **PostgreSQL** (Neon serverless) through a **SQL** data layer keyed on the Google OAuth sub, so a streak follows the player across devices.
- Wrote the **Python** generator (separate [open-source repo](#)) that uses **Google OR-Tools CP-SAT** as a uniqueness oracle, so every puzzle is provably **single-solution**, and scores difficulty by deduction depth.

Technologies: Next.js, React, TypeScript, Tailwind, PostgreSQL (Neon), SQL, Satori, Vercel; Python, OR-Tools CP-SAT, Pydantic, pytest, mypy

BundesPredict: Forecasting Service with a Bounded LLM Agent | [Website](#) | [Source Code](#)

Live, July 2026

- Built a Bundesliga match predictor on a **hand-written Dixon–Coles Poisson model** (weighted MLE via SciPy, time decay), **backtested walk-forward** over 1,800+ out-of-sample matches against the de-vigged market line.
- Integrated an **LLM API** (**Anthropic SDK**) as a **tool-calling agent** that turns free-text squad context into **bounded, server-clamped** adjustments, logged as typed, audited proposals; the deterministic model owns every published probability.
- Architected the Dockerized **FastAPI + PostgreSQL** backend: a relational schema under **Alembic migrations**, idempotent **SQL upserts** (ON CONFLICT) for scheduled ingest from third-party feeds, and a weekly automated refit-and-redeploy job, gated by **GitHub Actions CI/CD** that replays recorded LLM transcripts, so CI needs no API key.

Technologies: Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, Anthropic SDK, NumPy, SciPy, pandas, Next.js, Docker, GitHub Actions

Deximon: Pokémon TCG Collector Platform | [Website](#) | [Source Code](#)

Live, Summer 2026 • 6-Person Team

- Shipped a collector platform (digital binder, card-recognition scanner, marketplace) as the **largest contributor on a 6-person team** with **66 of 140 commits**, through **GitHub pull requests** with **CI** on every PR.
- Built binder persistence end to end, from the **React/Next.js** UI through a **FastAPI REST API** to a normalized **PostgreSQL** schema (**unique** and **check constraints**, foreign keys), in a five-service **Docker Compose** topology with an **AWS Rekognition scanner microservice** and **JWT** auth.
- Reviewed 8 merged teammate pull requests** and shipped follow-up fixes, root-causing an **N+1 query** on the marketplace feed (**SQLAlchemy** eager loading), a WebSocket handler that crashed on dead sockets, and multi-tab notification bugs.

Technologies: Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, TypeScript, React, Next.js, Docker Compose, WebSockets, AWS Rekognition, JWT, OpenAPI

OverTerm: Developer Tool for AI Coding Agents | [Source Code](#)

v1.0.4, September 2026 • Open Source

- Built and shipped a native macOS terminal in **Rust + Tauri v2** that detects whether an **AI coding agent** is running, finished, or waiting, from a **vt100 screen model** of the pseudoterminal that works on any CLI tool, plus exact turn events read from Claude Code hooks.
- Root-caused two bugs filed with the wrong diagnosis by reproducing them in the running app: a no-op window resize that made macOS drop keyboard focus, and a global **write()** that sent every terminal's replies to the active tab.
- Released it as a **one-command install** behind a tag-triggered **GitHub Actions** pipeline that verifies the signature and auto-bumps the Homebrew cask; merged 5 pull requests from 2 contributors.

Technologies: Rust, Tauri v2, TypeScript, xterm.js, portable-pty, vt100, objc2, GitHub Actions, Homebrew

Technical Skills

Languages: Python, TypeScript, JavaScript, Java, SQL, Rust, HTML/CSS

Databases: PostgreSQL, SQL (joins, transactions, constraints, indexing, upserts), schema design, Alembic migrations, SQLAlchemy, MySQL

Web & APIs: React, Next.js, FastAPI, Django REST Framework, REST APIs, OpenAPI, microservices, OAuth 2.0, JWT, WebSockets

AI & ML: LLM APIs (Anthropic SDK), tool-calling agents, prompt engineering, PyTorch, scikit-learn, NLP, RAG, statistical modeling (MLE, backtesting), NumPy, SciPy, pandas, OR-Tools, Claude Code

Cloud & DevOps: Git, GitHub Actions CI/CD, Docker, Docker Compose, AWS, Vercel, Railway | **Practices:** pytest, JUnit, Agile, code review